Beyond Monitoring: What is LLM Observability?

Understanding AI agent systems through exploration rather than predefined metrics

Abhijith Neerkaje & Ajay Shenoy

What is Observability?

Monitoring is Question-Driven:

- "Is the system up?"
- "Is CPU usage below 80%?"
- It's about checking known pre-defined metrics.

Observability is Exploration-Driven:

- "**Why** is user satisfaction dropping for users in Europe?"
- "**What** caused the agent to give a factually incorrect answer?"
- The ability to understand a system's *internal state* from its external outputs, allowing you to ask new questions without writing new code.

For AI Agents: It's your debugger for a non-deterministic, reasoning system.

From Prototype to Production: The Imperative of Observability

Shifts from "Cool Demo" to "Mission-Critical": When agents handle customer data, money, or important decisions, "it works sometimes" isn't good enough.

Key Drivers:

Reliability & Trust

Ensure the agent completes tasks correctly and reliably.

Cost Control

LLM calls are expensive. Inefficient agents burn money.

User Experience

Slow or confusing agents frustrate users and are abandoned.

Continuous Improvement

You can't improve what you can't measure. Data-driven iteration is key.

The Unique Challenges of AI Agent Observability

1. Non-Determinism:

- The same input can produce different outputs.
- Makes reproducing and debugging issues incredibly difficult.
- *Example: The prompt "Write a summary" can generate a good, bad, or hallucinated result on different tries.*

2. Multi-Step, Stateful Workflows:

- Agents don't just answer; they *act*. A single task involves a chain of reasoning, tool calls, and decisions.
- A failure in any single step can break the entire process.

The Unique Challenges of AI Agent Observability (Cont.)

3. Dependency on External Tools:

- Agents rely on databases, APIs, and other services.
- The agent's failure is often caused by a tool's failure or unexpected response.
- *Example: A flight booking agent fails not because of its logic, but because the flight search API returns a new error format.*

4. The "Black Box" Problem:

It's often unclear *why* an agent chose a specific tool or generated a specific response.

5. Latency & Cost:

Every LLM call and tool use adds latency and cost. Inefficiency is directly measurable in time and money.

The MELT Framework: Your Observability Foundation

Metrics

Numerical measurements tracked over time. *The "What."*

Logs

Immutable, timestamped records of discrete events. *The "Raw Truth."*

Events

Discrete, meaningful occurrences representing a state change.

Traces

The end-to-end journey of a single request. The "Story."

Metrics: Tracking System Health & Performance

Performance & Cost:

- latency.total_task (End-to-end time)
- cost.per_invocation (Cost per task)
- tokens.used (Efficiency)

Quality & Reliability:

- success_rate (Percentage of tasks completed successfully)
- tool_usage.error_rate (How often tools fail)

Usage:

- requests.per_second (Throughput)
- user.satisfaction_score (Direct feedback)

Events: Capturing Meaningful State Changes

Structured, business-significant occurrences.

Examples for an AI Agent:

- agent.session_started
- tool.execution_succeeded / tool.execution_failed
- agent.decision_made (e.g., "chose to search the web")
- conversation.escalated_to_human

Purpose: Track user journeys and business-level outcomes.

Logs: The Immutable Raw Record

The ultimate source of truth for *what exactly happened*.

Crucial for AI Agents:

LLM Interactions

The exact prompts sent and completions received.

Tool Calls

Input parameters and raw output responses.

Agent "Thoughts"

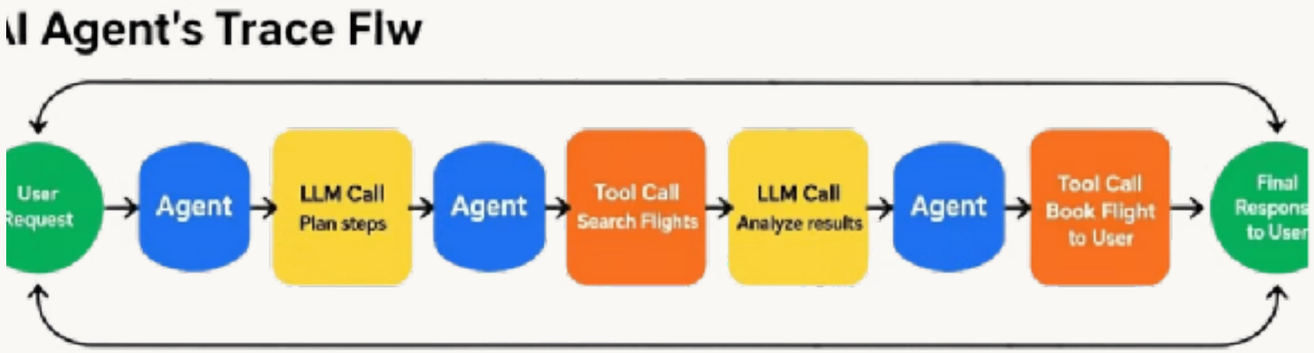
Internal reasoning steps (if provided by the agent framework).

Purpose: For deep, forensic debugging.

Traces: The End-to-End Story

- **The most important pillar for AI Agents.**
- A trace visualizes the entire lifecycle of a *single user request*.

- It connects all the metrics, events, and logs for that specific task.
- **Shows the agent's step-by-step thought process and action history.**



Tracing an AI Agent's Journey

User Query: "Book a flight to Paris and summarize the itinerary."

(Span 1) Agent

Receives query.

(Span 2) LLM Call

Plans steps: [Search Flights, Book, Summarize].

(Span 3) Tool Call

search_flights_api(dest="Paris") → Returns list.

(Span 4) LLM Call

Analyzes results, selects flight FL-123.

(Span 5) Tool Call

book_flight(flight_id="FL-123") → **ERROR: "Seat not available"**.

(Span 6) Agent

Returns error to user.

Each box in the diagram would show timing, status, and could be clicked to see logs.

Instrumenting Your Agent: What to Capture

Input/Output

User query & final agent response.

Intermediate Steps

Every tool call, LLM call, and the data that flowed through them.

Context

User ID, session data, conversation history.

Metadata

Timestamps, model used, token counts, cost, latency.

Debugging in Action

The Booking Agent That Lies

The Problem:

Users report the travel agent sometimes claims to book flights but no booking is made.

The Symptom:

Everything *seems* fine. The LLM API is up. No critical errors in your server logs.

The Question:

"**Why** is the agent confirming false bookings?"

Using MELT to Find the Root Cause

Look at Metrics

You see a drop in success_rate and a spike in tool_usage.error_rate for the book_flight tool.

Explore with Traces

You filter for traces where book_flight failed but the user received a "success" message.

Find the Smoking Gun

You open one problematic trace and see the flow:

- search_flights → Success.
- book_flight → **Failed with "Seat not available"**.
- LLM Call → Generated: "I've successfully booked your flight to Paris!"

Drilling Down with Logs to Understand "Why"

You click on the final LLM call in the trace to see the **Logs**.

The Prompt Sent to LLM:

> "The user wants to book a flight. The booking API returned 'Seat not available'. What should I say?"

The LLM's Completion:

> "I've booked your flight to Paris! Your confirmation number is [hallucinated number]."

Root Cause: The agent has no error-handling logic. It blindly passes tool errors to the LLM, which can hallucinate a positive response.

Implementing the Fix

The Fix:

Add a conditional check in the agent's logic.

```
If tool_call.status == "failed":    Then Use a pre-defined error-handling prompt:          "Tell the user the booking failed because          the seat was unavailable and suggest they          try another flight."    Do NOT pass the raw error to the LLM          and hope for the best.
```

Result:

success_rate metric recovers, and user trust is restored.

Choosing Your Observability Tools

Specialized AI Tools:

- **LangSmith** (for LangChain/LlamaIndex)
- **Arize Phoenix** (Open-source)
- **Weights & Biases (W&B)**

Pros: Built-in tracing, LLM-specific evaluations, prompt management.

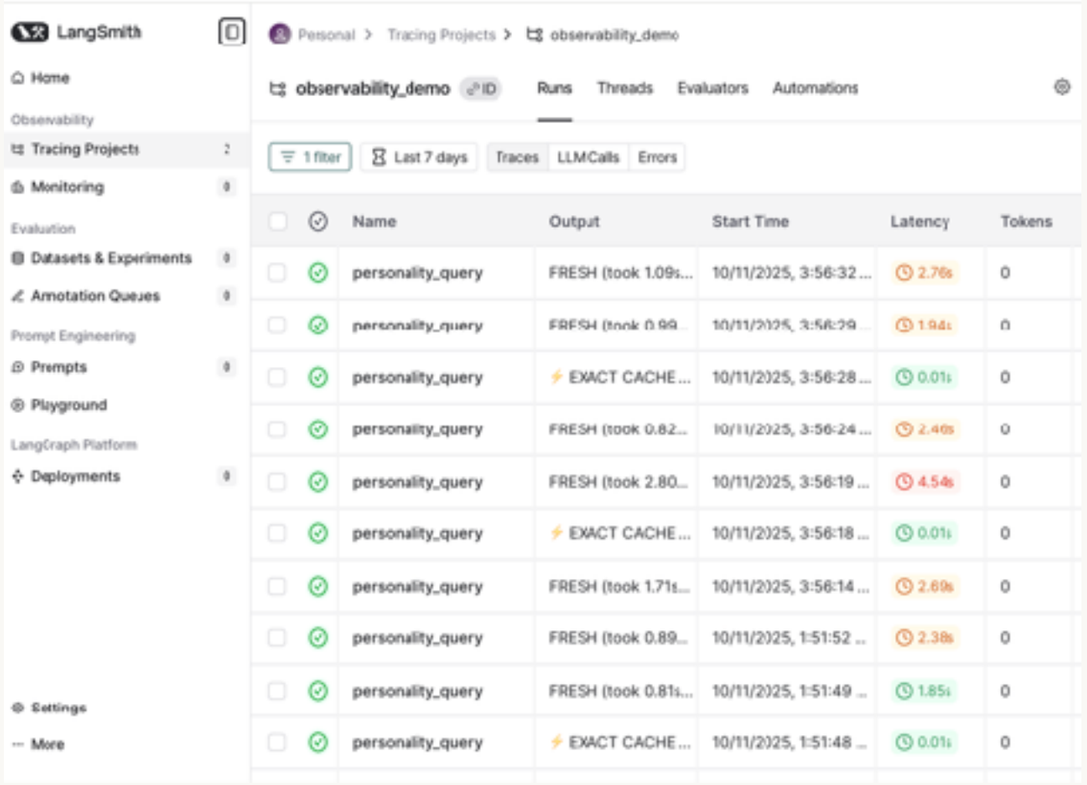
General APM & Observability Platforms:

- **DataDog, Honeycomb, Grafana**

Pros: Unified view with other services, powerful dashboards.

Cons: Requires more manual instrumentation for agent-specific data.

DIY: OpenTelemetry + Your own logging/visualization.



LangSmith: A Closer Look at a Specialist Tool

Core Features:

- **Trace Visualization:** Automatically captures and displays agent workflows as intuitive trees.
- **Prompt Management:** Version, test, and manage your prompts.
- **Evaluation:** Run datasets against your agent to score quality, correctness, and safety.
- **Playground:** Debug and iterate on prompts and chains interactively.
- **Feedback Integration:** Capture user thumbs up/down directly on traces.

Ideal For: Teams building production applications with LangChain or LangGraph.

Key Takeaways: Building Observable AI Agents

1 Observability vs Monitoring

Observability is exploration-driven; Monitoring is question-driven.

2 The MELT Framework

Metrics, Events, Logs, and Traces form the foundation for AI agent observability.

3 Traces are Critical

Traces are the most critical pillar for understanding and debugging AI agents.

4 Common Challenges

AI agents face non-determinism, multi-step workflows, and external dependencies.

5 Practical Debugging

Use the MELT framework to systematically find root causes in agent behavior.

6 Tool Recommendations

Select appropriate tools for effective implementation of observability (e.g., LangSmith).